Copyright © 2022. Facet Publishing. All rights reserved.

9 Digital Technologies and Data Systems

> While computing technologies are young by comparison with other efforts of human ingenuity, their origins go back to many early chapters in the human quest for the understanding and realization of mechanical aids to knowledge.
>
> Giuseppi Primiero (2020, 7)

> Computing is symbol processing. Any automaton capable of processing symbol structures is a computer . . . We may choose to call such symbol structures information, data, or knowledge, depending in the particular 'culture' within computer science to which we belong.
>
> Subrata Dasgupta (2016, 121)

> Data science is the process by which the power of data is realised, it is how we find actionable insights from among the swathes of data that are available.
>
> David Stuart (2020, xvi)

Introduction

Technology, from the Greek *techné*, meaning art, skill or craft, is usually taken to mean the understanding of how to use tools, in the broadest sense of the word. The term *information technology* was first used in the 1950s to describe the application of mechanised documentation and then-new digital computers, and became widely used in the 1980s to describe the wider-spread use of digital technology (Zorkoczy, 1982).

Information technology is usually associated with computers and networks. But, in a wider sense stemming from the original meaning of the word, the technologies of information include all the tools and machines which have been used to assist the creation and dissemination of information throughout history, as discussed in Chapter 2; from ink and paper, through printing to microforms, document reproduction and photocopying, and mechanised documentation technologies such as card indexes, punched cards, edge-notched cards, and optical coincidence cards. Krajewski (2011)

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

examines the idea of card index files as a 'universal paper machine', in a sense the forerunner of the computer, between the 16th and 20th centuries.

Our focus here is on digital technologies, echoing the view expressed by Gilster (1997) that all information today is digital, has been digital or may be digital. This chapter covers the basics of digital technologies and the handling of data; the following chapter deals with information systems.

Digital technologies

We will describe these aspects only in outline; see Ince (2011) and Dasgupta (2016) for more detailed but accessible accounts, and Primiero (2020) for a more advanced summary; for accounts of the historical development of the computer, see Ceruzzi (2012) and Haigh and Ceruzzi (2021).

Any digital device represents data in the form of *binary digits* or *bits*. Patterns of bits, *bit strings* with bits conventionally represented by 0 and 1, may represent data or instructions. A collection of eight bits is known as a *byte*. Quantities of data are represented as multiples of bytes, for example: a kilobyte, defined as 2^{10} bytes, i.e. 1024 bytes.

Any character or symbol may be represented by a bit string, but this requires an agreed coding. The most widely used code since the beginning of the computer age was the American National Standards Institute's ASCII (American Standard Code for Information Interchange) 7-bit code, but it is limited to the Latin alphabet, Arabic numerals and a few other symbols. It has been supplanted by Unicode, which can handle a wider variety of symbols and scripts. It can do this by using a 32-bit code; the more bits in the code, the more different symbols can be coded. Codes provide arbitrary representations of characters; for example, the ASCII code for the letter L is the bit string 1001100.

The idea that a general-purpose computing machine might be possible stems from the theoretical studies of the British mathematician Alan Turing (1912–54), whose concept of the *Turing machine*, developed in the 1930s, showed that a single machine limited to a few simple operations (read, write, delete, compare) could undertake any task if appropriately programmed.

The basic architecture for a practical digital computer was set out by John von Neumann (1903–57), the Hungarian-American mathematician, in the 1940s (Figure 9.1). His design, which gave the first formal description of a single-memory stored-program computer, is shown in Figure 9.2.

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

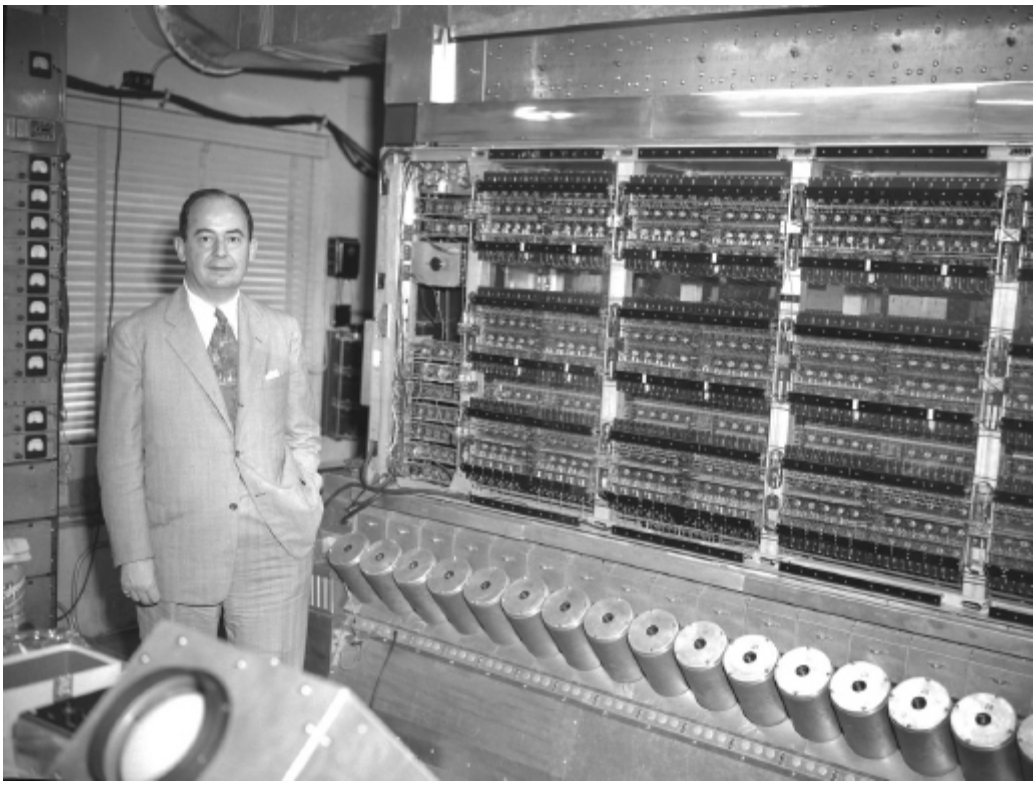

Figure 9.1 *John von Neumann* (Alan Richards photographer. From the Shelby White and Leon Levy Archives Center, Institute for Advanced Study, Princeton, NJ, USA)

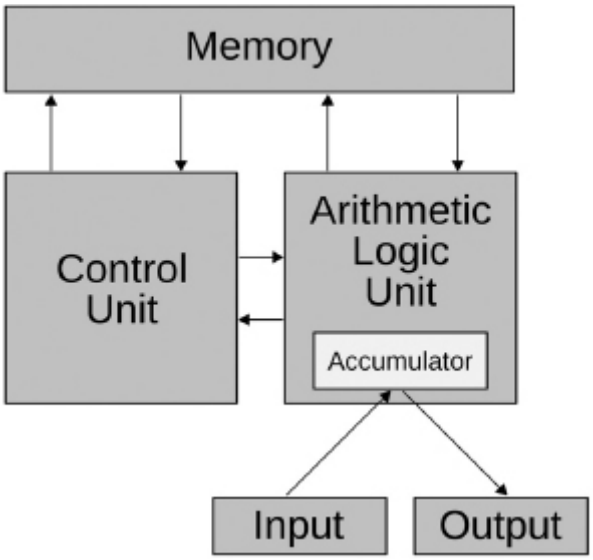

Figure 9.2 *Von Neumann computer architecture* (Wikimedia Commons, CC BY-SA)

This architecture is general purpose, in the sense that it can run a variety of programs. This distinguishes it from special-purpose digital computers which carry out only one task, as in digital cameras, kitchen appliances, cars, etc. A von Neumann machine loads and runs programs as necessary, to accomplish very different tasks.

The heart of the computer, the *processor*, often referred to as the *central processing unit* (CPU), carries out a set of very basic arithmetical and logical operations with instructions and data pulled in from the *memory*, also referred to as *main* or *working* memory. This sequence is referred to as

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

the *fetch–execute cycle*. Two components of the processor are sometimes distinguished: an *arithmetic and logic unit*, which carries out the operations, and a *control unit*, which governs the operations of the cycle.

While programs and data are being used they are kept in the memory. While not being used they are stored long-term in *file storage*. Items in memory are accessible much more rapidly than items in file storage, but memory is more expensive, so computers have much more file storage than memory.

Data comes into the computer through its *input devices* and is sent into the outside world through the *output devices*. The components are linked together through circuits usually denoted as a *bus*, sometimes referred to more specifically as a *data bus* or *address bus*.

All of these components have undergone considerable change since the first computers were designed. Processor design has gone through three main technological stages. The *first generation* of computers used valves as processor components, and the *second generation* used transistors. Computers of the *third generation*, including all present-day computers, use circuits on 'silicon chips', by very-large-scale integration (VLSI), which allows millions of components to be placed on a single computer chip, thus increasing processing speed.

The storage elements of the computer's memory comprise regular arrays of silicon-based units, each holding a bit of data, and created using the same VLSI methods as processors. Earlier generations of computers used *core storage*, with data held on tiny, magnetised elements arranged in a three-dimensional lattice.

File storage, holding data and programs that are not needed immediately, has always used magnetic media, which can hold large volumes of data cheaply, provided that quick access is not required. Earlier generations of computers used a variety of tapes, drums and disks; current computers use *hard disks*, rapidly spinning magnetisable disks with moveable arms with *read /write heads*, to read data from, or write data to, any area of the disk. Regardless of the technology, each area holds one bit of information, according to its magnetic state.

Input devices fall into three categories: those which take input interactively from the user; those which accept data from other digital sources; and those which convert paper data into digital form. The first category comprises the venerable 'QWERTY' keyboard, originally developed for typewriters in the 1870s, together with more recently developed devices: the mouse and other pointing devices and the touch-screen. The second category comprises the silicon-memory data

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

stick, replacing the various forms of portable magnetic *floppy disks* used previously, and the ports and circuits by which the computer communicates with networked resources. The third category is that of the scanner, digitising print and images from paper sources.

Output devices are similarly categorised; display screens, which allow user interaction through visual and sound output; devices which output digital data, such as data sticks, network circuits and ports; and devices which print paper output, typically laser or inkjet printers.

The most recent development is the trend towards mixed reality and immersive environments, which may require specific input/output devices (Greengard, 2019; Pangilinan, Lukos and Mohan, 2019). *Virtual reality* (VR) provides immersive experiences in a computer-generated world, often using head-mounted display devices, such as Oculus Rift. *Augmented reality* (AR) overlays computer-generated information, often visual but sometimes multisensory, on the real world. *Mixed reality* is the combination of real and virtual environments, a spectrum with AR at one end and VR at the other. These are of increasing relevance to the provision of information services (Robinson, 2015; Varnum, 2019; Dahya et al., 2021; Kroski, 2021).

Networks

Virtually all computers are connected via some form of network to others, to enable communication and information access and sharing. Since the 1990s the internet and the World Wide Web have become ubiquitous, such that it has become difficult to think about digital technologies without these at centre stage.

The growth of networked computing has been driven by three factors: communications technology, software and standards. In terms of technology, older forms of wired network, originating in the twisted copper-pair cables used for telegraph systems from the mid-19th century, have been succeeded by fibre-optic cables and various forms of wireless transmission. These allow much faster transmission speeds and greater information-carrying capacity; such systems are described loosely as *broadband*. Software systems have improved the efficiency and reliability of transmission greatly: an important example is *packet switching*, by which messages may be split up and their constituents sent by the fastest available route, being recombined before delivery.

Standards are fundamentally important, so that different forms of network, and different kinds of computers connected to them, can communicate effectively. The internet, originating in 1960s

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

networks built for defence research in the USA, is a worldwide network of networks, integrated by common standards: the internet control protocols, commonly known as TCP/IP (Transmission Control Protocol/Internet Protocol).

The World Wide Web, often spoken of as if it were synonymous with the internet, is in fact a major internet application. A system for allowing access to interlinked hypertext documents stored on networked computers, it originated in the work of Sir Tim Berners-Lee at CERN, the European nuclear research establishment. Berners-Lee introduced the idea in 1989 in an internal memorandum with the modest title 'Information management: a proposal'.

The web is based on *client–server* architecture: a web browser, the client, on the user's computer accesses the website on the remote server computer, through the internet, and downloads the required web page. This relies on a number of standards. For example, web pages must be created using a mark-up language, typically HTML, sometimes referred to as the *lingua franca* of the internet, and be identified by a Uniform Resource Identifier, commonly referred to as a Uniform Resource Locator or URL; a Hypertext Transfer Protocol (HTTP) enables communication between client and server.

The *Internet of Things* (Greengard, 2021) is a generic term for the aggregation of network-enabled 'smart' devices of all kinds, able to collect and share data.

The internet and web, despite their great reach and influence, are by no means the only significant computer networks. Many private, local and regional networks exist, generally using the standards of the internet and web for compatibility.

Software

Software, in its various forms, provides the instructions for digital computers to carry out their tasks and can be categorised into *systems software* (firmware and operating systems) and *application software*; for short overviews, see Ince (2011), White and Downs (2015) and Dasgupta (2016).

Firmware, such as BIOS (Basic Input Output System), EFI (Extensible Firmware Interface) and UEFI (Unified Extensible Firmware Interface), comes pre-installed on all computers and is activated when the computer is powered up, to initialise the hardware and start the operating system.

The *operating system*, specific to a particular type of computer, controls the hardware, allocates system resources, such as memory, and runs applications. Users interact with the operating system to manage files, use input and output devices, and run applications. Examples of operating systems are Windows, Unix and several variants of Unix, including Linux and Mac OS.

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

Users interact with the operating system via a program termed a *shell* because it forms an outer layer around an operating system; examples of shells are Bash, zsh and Powershell. Most users will rely on a graphical user interface to the shell for convenience. An alternative form of interface uses a *command line*, with the user typing instructions (such interfaces are commonly referred to as shells, although strictly the shell is the program to which they are the interface). This type of interface offers a more direct control over computer operations than with a graphical interface, but requires the user to understand the set of commands and the syntax, and to have some understanding of the workings of the operating system.

Applications software, discrete programs for carrying out specific tasks, for a specific purpose, may either be installed on individual computers, as, for example, word processors, spreadsheets, and web browsers typically are, or may be accessed on the web. The latter is usual for search engines, library management systems, and social media platforms. From the users' perspectives, application programs cause the computer to carry out tasks defined at a fairly high level: search the bibliographic database for authors with this name; calculate the mean of this column of figures; insert this image into this blog post; and so on.

All software must be written in a programming language, though this will be invisible to the user, who will have no reason to know what language any particular application is written in. At the risk of over-simplification, we can say there are four kinds of software language: low-level and high-level programming languages, scripting languages and mark-up languages.

Low-level programming languages, also referred to as *assembly* languages or *machine code*, encode instructions at the very detailed level of processor operations; these languages are therefore necessarily specific to a particular type of computer. This kind of programming is complex and difficult, but results in very efficient operation; it is reserved for situations where reliably fast processing is essential.

High-level programming languages express instructions in terms closer to user intentions, and are converted by other software systems, *compilers* and *interpreters* into processor instructions in machine code for a particular hardware. Programs are therefore much easier to write, understand and modify than those in machine code. Such languages can be used on different types of computer. There have been many such languages, some general purpose and some aimed at a particular kind of application. The first examples were developed in the 1950s, and two of the earliest are still in use: FORTRAN, still a language of choice for scientific and engineering applications, and COBOL, still used in many legacy systems for business applications. Currently popular languages are Java, C++, R and Python. Compilers convert a whole program into machine

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

code, whereas interpreters convert one line at time. Programming languages are described as either *compiled*, for example C++, or *interpreted*, for example Python.

Scripting languages are a particularly significant form of high-level language, designed to support interaction, particularly with web resources; for example, to update a web page in response to a user's input rather than reloading the page each time, or to create a *mashup*, an integration of data from several web sources. Examples are JavaScript, PHP and Perl.

Mark-up languages are somewhat different in nature, as they are designed to annotate text so as to denote either structural elements ('this is an author name'), presentation ('print this in italic') or both. The first such languages were designed for formatting documents for printing; an example is LaTeX. More recent examples control the format and structure of web resources; examples are HTML and XML.

Regardless of the language used, software is created in essentially the same way. The abstract process to be followed, the *algorithm*, is determined and its steps are converted into statements in an appropriate language; this is the process of *coding* or *programming*. For small-scale software development, perhaps for a piece of code to be used temporarily or just by one person, writing the code and testing that it works correctly is sufficient. For a major piece of software, other stages will include design, testing and maintenance; these stages, together with the coding stages, make up *system development* or *software engineering.* For accessible introductions to these topics, see Louridas (2020) and Montfort (2021).

One influence on the way software is created is the availability of libraries of *subroutines*, packages of code in a particular language to undertake specific tasks which can be included in programs by a single call. If a *subroutine library* (also called a *code library*) is available for a particular topic, it means that a coder can create a program very quickly and economically by a series of calls to subroutines, without having to write the code in detail; see Wintjen (2020) for detailed examples. There are an increasing number of subroutine libraries available for topics of relevance to the information disciplines. For example, the Python language has libraries of subroutines for text processing, for extracting data from URLs, for extracting data from mark-up languages such as HTML and XML and for processing data from Wikipedia.

An innovation in the way software is provided has come from the *open source* movement. Commercial software is usually provided as a 'black box'; the user has no access to the program code and therefore cannot modify the system at all, nor even know exactly how it works. With open source software, the user is given the full source code – the original programs – which they are free to modify. This makes it easy to customise software to meet local needs and preferences. It

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

also allows users to collaborate in extending and improving the systems, correcting errors, etc. Most such software is free, leading it to be known as FOSS (Free and Open Source Software). Well-known examples of full FOSS systems relevant to library and information management are the Koha and FOLIO-ERM library management systems, the EPrints repository system, the Drupal content management system and the Open Journal Systems (OJS) e-journal publishing system; see, for example, Breeding (2017) and Choo and Pruett (2019). Many smaller-scale FOSS software products are available; the GitHub repository is the most commonly used means for making these available (Beer, 2017). Github is commonly integrated with the Jupyter notebook environment, which combines simple word processing with the ability to create and run programs; this integration allows for testing, sharing and distribution of open source software and is widely used for data-wrangling functionality, including library/information applications (Wintjen, 2020).

There are a number of reasons why information professionals may wish to do their own coding, particularly now that relevant code libraries make it so much quicker and easier to do so. The main reasons would be to customise a procedure to a specific situation, to merge data from different sources, to create tidy, consistent data or to link systems and data together in a particular context.

Artificial intelligence

Artificial intelligence (AI) encompasses a complex array of concepts and processes, directed to making it possible for digital computers to do the kind of things that minds can do; see Boden (2016), Cantwell Smith (2019) and Mitchell (2019) for accessible overviews, Floridi (2019) for ideas of its future and Mitchell (2021) for caveats about how much we can expect from AI, especially with regard to endowing machines with common knowledge and common sense. It is ‘a growing resource of interactive, autonomous, self-learning agency, which enables computational artefacts to perform tasks that otherwise would require human intelligence to be carried out successfully’ (Taddeo and Floridi, 2018, 751).

Although AI as such has been possible only since the middle of the 20th century, it has been preceded by several centuries of interest in mechanical automata, game-playing machines, calculating machines, automated reasoning and the like; see Pickover (2019) for an informal history. Although Ada Lovelace mused on a ‘calculus of the nervous system’, and the possibility of mechanical devices composing music, AI became a prospect only with the development of the digital computer.

Alan Turing’s 1950 article ‘Computing Machinery and Intelligence’ was the first serious philosophical investigation of the question ‘can machines think?’ In order to avoid the problem

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

that a machine might be doing what might be regarded as thinking but in a very different way from a human, Turing proposed that the question be replaced by the question as to whether machines could pass what is now known as the *Turing Test*. Turing proposed an 'Imitation Game', in which a judge would interact remotely with a human and a computer by typed questions and answers, and would decide which was the computer; if the judge could not reliably decide which was the computer, then the machine passed the test, as Turing believed that digital computers would be able to do. Turing's article also proposed for the first time how a machine might learn, and noted a series of objections to the idea of AI which are still raised today.

The term 'artificial intelligence' is attributed to John McCarthy (1927–2011) and appears in a proposal for a workshop at Dartmouth College, in which 'the artificial intelligence problem is taken to be that of making a machine behave in ways that would be called intelligent if a human were so behaving' (McCarthy et al., 1955). The workshop, held in 1956, established the topic as a field for realistic academic study (Nilsson, 2012). A fundamental insight was provided by another founder of AI, Marvin Minsky (1927–2016), who argued that intelligence was not the result of a single sophisticated agent or principle but, rather, the result of the interaction of many simple processes (Minsky, 1987). This suggested that AI might be achieved by the combination of relatively simple programs.

The first approaches to practical AI were centred on attempting to capture in a *logic-based* program the knowledge and reasoning processes of a human expert: a doctor diagnosing disease, or a chemist interpreting analytical data or planning the synthesis of new compounds. This symbolic approach to AI, now sometimes referred to as Good Old Fashioned AI (GOFAI), had some successes, but proved to be limited by the difficulty of entering the large amounts of knowledge, particularly general 'common sense' knowledge, required for even seemingly simple tasks.

A newer generation, or 'second wave', of AI systems relies on programs which enable machines to learn for themselves, given very large datasets and massive computing power. (Big Data is not necessarily a *sine qua non* for AI; Floridi (2019) notes that in some cases small datasets of high quality may be more valuable.) For example, in 2011, IBM's Watson AI program beat a human champion at the game Jeopardy, which requires general knowledge questions, including puns, jokes and cultural references, to be solved quickly, without access to any external resources. Watson's success was due to massive computing power using probabilistic reasoning, plus extensive resources, such as the whole of Wikipedia and many other reference works, held in computer memory. In 2017 the program AlphaZero was given only the rules of chess as a starting point, and trained its neural networks by playing against itself many millions of times. Within hours

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

it was at a level to compete with the best chess programs. This 'brute force' approach has led to greater success, but at the cost that it may be impossible to understand how and why the system is working; as Turing foresaw, machines may be 'thinking' but they do it differently. In particular, we should not attribute any element of 'understanding' to an AI because it can equal, or better, human performance at tasks which would require a human to understand a situation.

At the core of most modern AI systems is *machine learning*, a way of programming computers to learn from experience, using large sets of data for training (Alpaydin, 2016). This comes in three forms: *supervised* learning, in which the algorithms are given large sets of labelled examples; *unsupervised* learning, in which the algorithms discover patterns *de novo* in sets of unlabelled data; and *reinforcement* learning, in which the system receives feedback as to whether the change it made was good or bad. This approach encompasses the *neural net* data structure, a complex linkage of many simple processing units and *evolutionary* or *genetic* algorithms, which allows programs to develop themselves to improve their performance. So-called *deep learning* uses a neural net with many layers by which machines train themselves to perform tasks (Kelleher, 2019). This can lead to very high performance, at the cost of creating models of such complexity that it may be difficult to understand why the artificial agent is making its judgements. All machine-learning systems face such problems of explanation, and also problems of algorithmic bias if the original data is incorrect or biased in any way.

Although somewhat displaced by machine learning and statistical algorithms, research continues on knowledge representation, using data structures such as *frames* or *semantic nets*, so as to allow artificial agents to simulate human reasoning, applying logical deduction, application of rules, or probabilistic reasoning. This is incorporated in *expert systems*, dealing with one specific task, typically of diagnosis, classification, recommendation or prediction, in scientific, medical, legal and financial domains. The two approaches, symbolic and statistical, are beginning to be used together in systems combining the power of both (Boden, 2016).

AI has raised ethical concerns, initially because of fears about 'killer robots' and 'superhuman intelligences', more recently and realistically because of dangers of algorithmic bias (Coeckelbergh, 2020); a full discussion is given in Chapter 15.

Robotics, the design of machines to undertake tasks without direct human control, is linked with AI, since the machines must be programmed to undertake activities which would otherwise require the direction of an intelligent person. Indeed, a robot may be understood as an AI given autonomous agency, albeit usually of very limited scope, in the physical world. Agents on the internet which emulate a human user are termed *bots*, by analogy with robots in the physical

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

world; those which are able to simulate human conversation may be termed *chatbots*. The term 'robot' was first used in English in Karel Capek's play *R. U. R.* (*Rossum's Universal Robots*) of 1921, from Czech 'robata', meaning 'forced labour'. The use of robots is well established in many areas, including libraries and archives, typically undertaking repetitive tasks; the number of robots in use depends on the definition used, but is certainly in the millions.

Application of AI in libraries, archives and similar institutions is, for the most part, at an initial exploratory stage, but the possibilities are becoming evident; see Griffey (2019) and Rolan et al. (2019). They include metadata creation, information extraction and text and data mining.

Interfaces, interaction and information architecture

The ways in which people interact with computers, generally described under the headings of *human–computer interaction* (HCI), *human–information interaction*, *interaction design* or *usability* , is an area of evident importance to the information sciences. It is particularly concerned with the design and evaluation of interfaces to information systems of all kinds (Sharp, Preece and Rogers, 2019).

Information architecture is the organising and structuring of complex digital information spaces, typically on websites. It draws from principles of information design on the printed page, of HCI and of categorisation and taxonomy. The term was coined in the 1970s by Saul Wurman, originally a 'real' architect, who came to believe that the principles behind the design of buildings and physical places could be applicable to information spaces. Information architecture came into its own from the late 1990s, with the expansion of the web and the need to bring order and structure to web-based information. For overviews, see Rosenfeld, Morville and Arango (2015) and Arango (2018).

Data systems

The growth of data science, and the increased emphasis on managing and curating data, as distinct from information, has been a major influence on information science since 2010. For an accessible general introduction, see Kelleher and Tierney (2018), for an overview aimed at library /information applications, see Stuart (2020), and for a more detailed and practical treatment, see Shah (2020) and Wintjen (2020). For an analysis of the idea of data itself, see Hjørland (2020), and

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

for examples of data practices in science, including the issues discussed later in this chapter, see Leonelli and Tempini (2020). We discuss this under two main headings: data wrangling, the processes of acquiring, cleaning and storing data; and techniques for finding meaning in data.

Data wrangling

The term *data wrangling* is sometimes used to refer to all activities in the handling of data. We will consider them, and the digital tools which support them, under four headings: collecting; storing; cleaning; and combining and reusing. A useful overview from an information science perspective is given in Stuart (2020, chapter 4).

Collecting

Collecting data is most simply achieved by typing it in from scratch. If it already exists in printed form, with a recognisable structure, then optical character recognition (OCR) may be used, though this is rarely error-free, and entry must be followed by cleaning. If the data exists in some digital form, then it may be selected and copied; a tool such as OpenRefine may be useful in collecting and parsing data, and in converting between formats.

If the required data is to be found on one or more web pages, then a process of *web scraping* can be used to obtain it. This is the automatic extraction of information from websites, taking elements of unstructured website data and creating structured data from it. Web scraping can be done with code written for the purpose, or by using a browser extension such as Crawly or Parsehub. This is easier if an Applications Programming Interface (API) can be accessed. Described as ‘an interface for software rather than people’, an API in general is a part of a software system designed to be accessed and manipulated by other programs rather than by human users. Web APIs allow programs to extract structured data from a web page using a formal syntax for queries and a standard structure for output; see Lynch, Gibson and Han (2020) for a metadata aggregation example.

Storing

Data may be stored in a wide variety of formats and systems. These include database systems such as MySQL, Oracle and Microsoft Access, and data-analysis packages such as SAS or SPSS. Most common is the spreadsheet, such as Microsoft Excel and Google Sheets, widely used for data entry and storage, with some facilities for analysis and visualisation of data. All kinds of structured data and text are stored in a two-dimensional row–column arrangement of cells, each cell containing

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

one data element. Consistency is of great importance, in the codes and names used for variables, the format for dates, etc., and can be supported by a *data dictionary*, a separate file stating the name and type of each data element (Broman and Woo, 2018).

Exchange formats are relatively simple, used to move data between more complex formats or systems. One widely used example is the CSV (Comma Separated Values) format. It holds data in plain text form as a series of values separated by commas in a series of lines, the values and lines being equivalent to the cells and rows of a spreadsheet. JSON (Javascript Object Notation) is a more complex XML-like format used for the same purposes; it can represent diverse data structures and relationships, rather than CSV's two-dimensional flat files.

Data collections are typically bedevilled by factors such as missing data, duplicate data records, multiple values in a cell, meaningless values and inconsistent data presentation.

Cleaning

Data-cleaning processes are applied to create *tidy data* as opposed to *messy data.* For a data sheet, tidy data requires that: each item is in a row; each variable is in a column; each value has its own cell; there are no empty cells; and the values of all variables are consistent.

To achieve this, data may be scanned by eye, and errors corrected manually. An automated process is more efficient and less error prone. Examples are the use of Python scripts for identification of place names in collections of historical documents (Won, Murrieta-Flores and Martins, 2018) and for the normalisation of Latin place names in a catalogue of rare books (Davis, 2020); see Walsh (2021) for a variety of examples. Data-handling packages, such SAS and SPSS, have their own data-cleaning functions.

One specific data-cleaning tool has been much used by library/information practitioners, as well as by scholars in the digital humanities. OpenRefine, a tool designed for improving messy data, provides a consistent approach to automatically identifying and correcting errors, grouping similar items so inconsistencies can be readily spotted by eye and identifying outliers, and hence possible errors, in numeric values. Examples of its use include the cleaning of metadata for museum collections (van Hooland et al., 2013) and extraction in a consistent form of proper names from descriptive metadata (van Hooland et al., 2015).

The most precise means of algorithmic data cleaning uses Regular Expressions (RegEx), a subset of a formal language for describing patterns in text, which ensures consistency. They were devised

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

originally within theoretical computer science by the American mathematician Stephen Kleen in the 1950s, using the mathematics of set theory as a way of formally describing all the operations that can be performed by computers in handling text.

In practice, regular expressions are search strings with special characters to denote exactly what is wanted. Examples of these special characters are:

- . matches any character ('wildcard')
- [] matches any of the characters within the brackets
- ^ matches beginning of a line of text

These expressions are used primarily for searching, parsing, cleaning and processing text, and also for the formal modelling of documents and databases; see applications to inconsistencies in bibliographic records (Monaco, 2020), errors in DOIs (Xu et al., 2019), metadata aggregation (Lynch, Gibson and Han, 2020) and identification of locations in the text of journal articles (Karl, 2019).

Combining and reusing

Data in digital form offers the possibilities of combining and integrating different formats and media. The results are sometimes described as *mashups*, usually meaning data from more than one source displayed in a single view. Data may also be reused for different purposes. The term *remix* is used for the amending or repurposing of data to create something novel; for examples, see Engard (2014). These processes may offer a reimagining of a dataset or collection of datasets so as to show different perspectives, and have the potential of creating new knowledge.

Finding meaning in data

One important strand of data handling in the information sciences is to analyse datasets so as to find meaning in them. We will outline the methods used in four broad and over-lapping themes: data mining and statistical analysis; data visualisation; text visualisation; and text mining. For more detailed treatment of this area, see Shah (2020) and Wintjen (2020).

Data mining and statistical analysis

Data mining and statistical analysis identifies nuggets of value in large volumes of unhelpful data (hence the mining analogy) and tests the significance of what is found by statistical methods. This is most commonly carried out using specialist subroutine libraries with the Python and R programming languages. A variety of techniques may be used, including: association, correlation

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

and regression; classification and clustering; analysis of variance and model building; and time series. When dealing with large sets of data, and typically investigating many possible correlations and interactions, it is essential to have a thorough understanding of statistical significance, ways to guard against misleading and chance findings.

Data visualisation

Data visualisation is now easier than ever before, but the results are not always pleasing or informative. There are many ways of achieving this, including routines in coding libraries, functions in spreadsheet and database software and specialist software packages. The ease of creating a visualisation means that the emphasis is now on representing the data in an understandable way, free from bias and distortion. See Tufte's classic 1990 book, Healy (2019), Kirk (2019) and Dick (2020).

One useful form of visualisation of particular value for information science is the network diagram, displaying nodes, with their size, and links, with their strength. There are numerous software packages available for this purpose, such as the NodeXL extension to Microsoft Excel (Ahmed and Lugovic, 2019), and packages such as VOSviewer and CitNetExplorer, designed for analysis and display of citation networks in published literature (Eck and Waltman, 2017; Williams, 2020).

Text visualisation

There are many ways of visualising textual data; a survey by Linnaeus University identified 440 distinct techniques (Kucher and Kerren, 2015). Few, however, have gained wide use.

The most common way of visualising text documents or collections of documents is the *word cloud*, a relatively simple tool for producing images comprising the words of the text, with greater prominence given to the words that appear most frequently. They are popular for giving quickly assimilated summaries and comparisons of the contents of lengthy documents, but have been criticised for giving overly simplistic views of complex data. The same is true of other simple text-visualisation methods, such as the *termsberry* and the *streamgraph*. Some suggestions for improvement by manipulating the layout of word clouds to make them more meaningful are given by Hearst et al. (2020).

Text mining

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

Text mining involves analysing and displaying the structure of texts. There are a variety of means to achieve this. Coding with specialised libraries, such as the Python language's TextBlob, gives greatest flexibility. General-purpose qualitative analysis packages, such as nVivo and MAXQDA, can be used for text mining, but are complex solutions to what may be only a simple need. Web-based environments for analysis, visualisation and distant reading of texts, such as Voyant Tools, provide a variety of analyses, including word clouds, word-frequency lists, common words shown in context, and occurrence of terms in different sections of a web document. Network analysis software such as VOSviewer can be used with bodies of text, producing visual displays of the frequency and co-occurrence of words and phrases. For library/information applications of this type of software, see Tokarz (2017), Miller (2018) and Hendrigan (2019). A more specialised tool, the *n-gram viewer*, allows the comparison of frequency of occurrence of words or phrases between datasets, or over time in a single dataset. The best-known example is Google Books Ngram Viewer, which provides a picture of how word, phrase and name occurrences have changed over time in Google's corpus of digitised books.

Summary

Digital technologies and data systems form the bedrock of the infosphere. An understanding of their nature and significance, and an ability to make practical use of relevant aspects, is essential for all information professionals.

- The technologies of information include all the tools and machines which have been used to assist the creation and dissemination of information throughout history.
- An understanding of the basics of computer and network architecture, coding, AI and HCI and the principles of data science is essential for all information professionals.

Key readings

Dasgupta, S. (2016) *Computer science: a very short introduction*, Oxford: Oxford University Press.

Ince, D. (2011) *The computer a very short introduction*, Oxford: Oxford University Press.

Shah, C. (2020) *A hands-on introduction to data science*, Cambridge: Cambridge University Press.

Stuart, D. (2020) *Practical data science for information professionals*, London: Facet Publishing.

References

Ahmed, W. and Lugovic, S. (2019) Social media analytics: analysis and visualisation of news diffusion using NodeXL, *Online Information Review*, **43** (1), 149–60.

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

Alpaydin, E. (2016) *Machine learning*, Cambridge MA: MIT Press.

Arango, A. (2018) *Living in information: responsible design for digital places*, New York: Two Waves Books.

Beer, B. (2017) *Introducing GitHub: a non-technical guide* (2nd edn), Sebastopol, CA: O'Reilly.

Boden, M. (2016) *AI: its nature and future*, Oxford: Oxford University Press.

Breeding, M. (2017) Koha: the original open source ILS, *Library Technology Reports*, **53** (6), 9–17.

Broman, K. W. and Woo, K. H. (2018) Data organization in spreadsheets, *American Statistician*, **72** (1), 2–10.

Cantwell Smith, B. (2019) *The promise of artificial intelligence: reckoning and judgement*, Cambridge MA: MIT Press.

Ceruzzi, P. E. (2012) *Computing: a concise history*, Cambridge MA: MIT Press.

Choo, N. and Pruett, J. A. (2019) The context and state of open source software adoption in US academic libraries, *Library Hi Tech*, **37** (4), 641–59.

Coeckelbergh, M. (2020) *AI ethics*, Cambridge MA: MIT Press.

Dahya, N., King, W. E., Lee, K. J. and Lee, J. H. (2021) Perceptions and experiences of virtual reality in public libraries, *Journal of Documentation*, **77** (3), 617–37.

Dasgupta, S. (2016) *Computer science: a very short introduction*, Oxford: Oxford University Press.

Davis, K. D. (2020) Leveraging the RBMS/BSC Latin places names file with Python, *Code4Lib Journal*, issue 48. Available at https://journal.code4lib.org/articles/15143.

Dick, M. (2020) *The infographic: A history of data graphics in news and communications*, Boston MA: MIT Press.

Eck, N. and Waltman, L. (2017) Citation-based clustering of publications using CitNetExplorer and VOSviewer, *Scientometrics*, **111** (2), 1053–70.

Engard, N. C. (2014) *More library mashups: exploring new ways to deliver library data*, London: Facet Publishing.

Floridi, L. (2019) What the near future of artificial intelligence could be, *Philosophy and Technology*, **32** (1), 1–15.

Gilster, P. (1997) *Digital literacy*, New York: Wiley.

Greengard, S. (2019) *Virtual reality*, Cambridge MA: The MIT Press.

Greengard, S. (2021) *The internet of things* (rev. edn), Cambridge MA: MIT Press.

Griffey, J. (ed.) (2019) Artificial intelligence and machine learning in libraries, *Library Technology Reports*, **55** (1), 1–29.

Haigh, T. and Ceruzzi, P. E. (2021) *A new history of modern computing*, Cambridge MA: MIT Press.

Healy, K. (2019) *Data visualization: a practical introduction*, Princeton NJ: Princeton University Press.

Hearst, M. A., Pedersen, E., Patil, L., Lee, E., Laskowski, P. and Franconeri, S. (2020) An evaluation of semantically grouped word cloud designs, *IEEE Transactions on Visualization and Computer Graphics*, **26** (9), 2748–61.

Hendrigan, H. (2019) Mixing digital humanities and applied science librarianship: using Voyant Tools to reveal word patterns in faculty research, *Issues in Science and Technology Librarianship*, **91**, 1–12.

Hjørland, B. (2020) Data (with big data and database semantics). In Hjørland, B. and Gnoli, C. (eds), *ISKO Encyclopedia of Knowledge Organization.* Available at www.isko.org/cyclo/data.

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

Ince, D. (2011) *The computer a very short introduction*, Oxford: Oxford University Press.

Karl, J. W. (2019) Mining location information from life- and earth-sciences studies to facilitate knowledge discovery, *Journal of Librarianship and Information Science*, **51** (4), 1007–21.

Kelleher, J. D. (2019) *Deep learning*, Cambridge MA: MIT Press.

Kelleher, J. D. and Tierney, B. (2018) *Data science*, Cambridge MA: MIT Press.

Kirk, A. (2019) *Data visualisation: a handbook for data driven design* (2nd edn), London: Sage.

Krajewski, M. (2011) *Paper machines: about cards and catalogs. 1548–1929*, Cambridge MA: MIT Press.

Kroski, E. (ed.) (2021) *32 virtual, augmented, and mixed reality programs for libraries*, Chicago: ALA Editions.

Kucher, K. and Kerren, A. (2015) Text visualization techniques: taxonomy, visual survey and community insights. Paper presented at the *IEEE Pacific Visualization Symposium.* Available at https://ieeexplore.ieee.org/document/7156366.

Leonelli, S. and Tempini, N. (2020) *Data journeys in the sciences*, Cham: Springer.

Louridas, P. (2020) *Algorithms*, Cambridge MA: MIT Press.

Lynch, J. D., Gibson, J. and Han, M. (2020) Analyzing and normalizing type metadata for a large aggregated digital library, *Code4Lib Journal*, issue 47. Available at https://journal.code4lib.org/articles/14995.

McCarthy, J., Minsky, M. L., Rochester, N. and Shannon, C. E. (1955) *A proposal for the Dartmouth summer research project on artificial intelligence*. Available at https://chsasank.github.io/classic_papers/darthmouth-artifical-intelligence-summer-resarch-proposal.html.

Miller, A. (2018) Text mining digital humanities projects: assessing content analysis capabilities of Voyant Tools, *Journal of Web Librarianship*, **12** (3), 169–97.

Minsky, M. (1987) *The society of mind*, London: Heinemann.

Mitchell, M. (2019) *Artificial intelligence. A guide for thinking humans*, London: Pelican.

Mitchell, M. (2021) Why AI is harder than we think. arXiv: 2104.12871. Available at https://arxiv.org/abs/2104.12871.

Monaco, M. (2020) Methods for in-sourcing authority control with MarcEdit, SQL, and Regular Expressions, *Journal of Library Metadata*, **20** (1), 1–27.

Montfort, N. (2021) *Exploratory programming for the arts and humanities* (2nd edn), Cambridge MA: MIT Press.

Nilsson, N. J. (2012) *John McCarthy 1927–2011: a biographical memoir*, Washington DC: National Academy of Sciences. Available at www.nasonline.org/publications/biographical-memoirs/memoir-pdfs/mccarthy-john.pdf.

Pangilinan, E., Lukos, S. and Mohan, V. (eds) (2019) *Creating augmented and virtual realities*, Sebastopol CA: O' Reilly.

Pickover C. A. (2019) *Artificial intelligence: an illustrated history*, New York: Stirling.

Primiero, G. (2020) *On the foundations of computing*, Oxford: Oxford University Press.

Robinson, L. (2015) Immersive information behaviour: using the documents of the future, *New Library World*, **116** (3–4), 112–21.

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.

Copyright © 2022. Facet Publishing. All rights reserved.

Rolan, G., Humphries, G., Jeffrey, L., Samaras, E., Antsoupova, T. and Stuart, K. (2019) More human than human? Artificial intelligence in the archive, *Archives and Manuscripts*, **47** (2), 179–203.

Rosenfeld, L., Morville, P. and Arango, J. (2015) *Information architecture: for the web and beyond*, Boston MA: O'Reilly.

Shah, C. (2020) *A hands-on introduction to data science*, Cambridge: Cambridge University Press.

Sharp, H., Preece, J. and Rogers, Y. (2019) *Interaction design: beyond human–computer interaction* (5th edn), Indianapolis: Wiley.

Stuart, D. (2020) *Practical data science for information professionals*, London: Facet Publishing.

Taddeo, M. and Floridi, L. (2018) How AI can be a force for good, *Science*, **361** (6404), 751–2.

Tokarz, R. E. (2017) Identifying criminal justice faculty research interests using Voyant and NVivo, *Behavioral and Social Sciences Librarian*, **36** (3), 113–21.

Tufte, E. R. (1990) *Envisioning information*, Cheshire CT: Graphics Press.

Turing, A. M. (1950) Computing machinery and intelligence, *Mind*, **49** (236), 433–60.

van Hooland, S., Verborgh, R., De Wilde, M. et al. (2013) Evaluating the success of vocabulary reconciliation for cultural heritage collections, *Journal of the American Society for Information Science and Technology*, **64** (3), 464–79.

van Hooland, S., De Wilde, M., Verborgh, R. et al. (2015) Exploring entity recognition and disambiguation for cultural heritage collection, *Digital Scholarship in the Humanities*, **30** (2), 262–79.

Varnum, K. J. (ed.) (2019) *Beyond reality: augmented, virtual, and mixed reality in the library*, Chicago: ALA Editions.

Walsh, S. (2021) *Between the spreadsheets: classifying and fixing dirty data*, London: Facet.

White, R. and Downs, T. E. (2015) *How computers work* (10th edn), Indianapolis IN: Que.

Williams, B. (2020) Dimensions and VOSviewer bibliometrics in the reference interview, *Code4Lib Journal*, issue 27. Available at https://journal.code4lib.org/articles/14964.

Wintjen, M. (2020) *Practical data analysis using Jupyter Notebook*, Birmingham: Packt.

Won, M., Murrieta-Flores, P. and Martins, B. (2018) Ensemble Named Entity Recognition (NER): evaluating NER tools in the identification of place names in historical corpora, *Frontiers in Digital Humanities*, 5.2. DOI: 10.3389/fdigh.2018.00002.

Xu, S., Hao, L., Zhai, D. and Peng, H. (2019) Types of DOI errors of cited references in Web of Science with a cleaning method, *Scientometrics*, **120** (3), 1427–37.

Zorkoczy, P. (1982) *Information technology: an introduction*, London: Pitman.

Introduction to Information Science, Facet Publishing, 2022. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=6707552.
Created from univ-people-ebooks on 2026-01-30 08:15:35.